CENG-567
REINFORCEMENT LEARNING

Lecture-1

Multi-armed Bandits

A/B/n Testing, Epsilon Greedy

Dr. Osman GÖKALP

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Everyday Examples of Bandit Problems

When you log in or shop online…

You see different **versions** of the same app (A/B testing)

Ads shown are **personalized** to your profile

Prices may **change dynamically** based on demand or user data

Common Thread:
All are modeled as **Multi-Armed Bandit (MAB)** problems to find optimal decisions.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

What Is a Multi-Armed Bandit (MAB)?

A **simplified Reinforcement Learning (RL)** problem

The agent makes a **single-step decision** (no future states)

Objective: **maximize immediate reward**

Must balance:
→ **Exploration** of new actions
→ **Exploitation** of known good actions

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The Exploration–Exploitation Trade-Off

Exploration: trying new actions to discover potentially higher rewards

Exploitation: choosing actions known to yield good outcomes

This trade-off is central to **all RL methods**

Business and AI systems must **optimize** this balance continuously

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case Study: Online Advertising

Scenario:
Which ad should we show to each visitor to maximize clicks?

MAB Component	Online Ad Example
Arm	Each ad version
Pull	Showing an ad
Reward	Click / no click

Used as a **running case study** for the MAB.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Scope of this Lecture

We focus on **non-contextual MABs** — (no user profile, time of day, etc.)

You'll learn:

The exploration–exploitation trade-off

What a MAB is

Case study: online advertising

Exploration strategies:

- **A/B/n testing**
- **ε-greedy actions**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The Exploration–Exploitation Trade-Off

Core idea:
RL is about learning *from experience* without explicit supervision.

The agent must:

Observe the **consequences** of its actions

Learn **which actions yield higher rewards**

Balance between **trying new actions** and **using known good ones**

Analogy:
Like learning how to study — you **experiment (explore)** and **stick to what works (exploit)**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Why Exploration Matters?

Exploration enables discovery:
Without it, we might **never find better methods**
Helps uncover **new strategies or unknown opportunities**

But:
Continuous exploration is **inefficient**
We must also **exploit** what we already know works

✅ **Goal:**
Find the **right balance** between exploration and exploitation for **efficient learning**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The Trade-Off in Reinforcement Learning

Exploration:

Trying actions with **uncertain outcomes**

Short-term loss, **possible long-term gain**

Exploitation:

Choosing the **best-known action**

Short-term gain, possible missed opportunities

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Why Focus on This Trade-Off in MAB Problems?

Even though the trade-off exists in all RL problems, it is **most clearly studied in MAB** because:

One-step RL:

- No future states — simplifies analysis of exploration methods
- Easier to compare and **prove performance mathematically**

Online learning nature:

- Agent learns and acts **in real time**
- Poor exploration = **real financial or operational cost**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Real-World Impact of Inefficient Exploration

In MAB (like online ads or pricing systems):

Each **bad decision** = **lost revenue** or user dissatisfaction

Efficient exploration becomes a **business-critical issue**

Key takeaway:
Balancing exploration and exploitation in MAB is not just theoretical — it's **economically essential**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Multi-Armed Bandit (MAB)

Core idea:
Identify the **best action** among several choices by learning through **trial and error**.

Examples:

Finding the best **website layout**

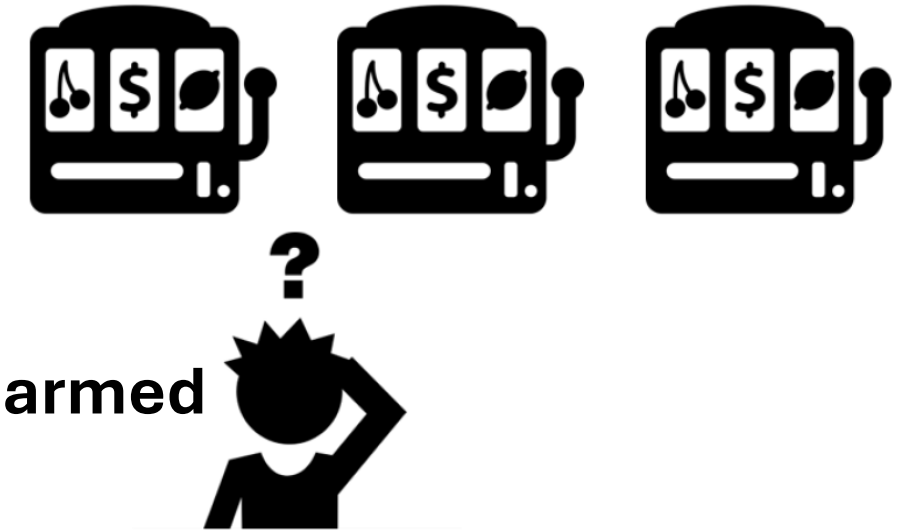
Selecting the most effective **ad banner**

Choosing the **highest-return strategy**

Focus:
We consider discrete actions — the **k-armed bandit** problem.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Origin of the Name



Analogy:
A gambler faces a row of **slot machines ("one-armed bandits")**.
Each machine:

- Gives a **random reward** drawn from its own probability distribution.
- **Looks identical**, but payout patterns differ.

Goal:
Maximize total reward by choosing **which machine to play** each turn.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The Gambler’s Dilemma

At every step, the gambler must decide:

- **Exploit**: play the machine with the highest observed average reward so far
- **Explore**: try a different machine to gather new information

Challenge:
Rewards are **stochastic** — a machine may appear best **just by chance**!

→ **Balance exploration and exploitation to find the truly best arm.**

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Characteristics of a MAB Problem

- ❑The agent takes sequential actions. After each action, a reward is received.
- ❑An action affects only the immediate reward, not the subsequent rewards.
- ❑There is no "state" in the system that changes with the actions that the agent takes.
- ❑There is no input that the agent uses to base its decisions on. That will come later in
- ❑We will discuss contextual bandits later.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

The Bandit Exercise

See '**Multi-armed bandits.ipynb**' file in the Teams course page.

1. We first, create a class (GaussianBandit) for a single slot machine that gives a reward from a normal (Gaussian) distribution with respect to a given mean and standard deviation.
2. Next, we create a class (GaussianBanditGame) that will simulate the game.
   - A game instance receives a list of slot machines as inputs. It then shuffles the
   - order of the slot machines so that you won't recognize which machine gives the
   - highest average reward. In each step, you will choose a machine and aim to get the
   - highest reward.
3. Then, we create some slot machines and a game instance.
4. Finally, play the first machine for a couple of rounds and observe at the end if it is the best machine.
   - (executing *game.bandits[0].mean*)

Case study – online advertising

Continue to see '**Multi-armed bandits.ipynb**' file in the Teams course page.

Scenario:
A company runs **digital banner ads** across various websites to promote a product.

Goal:
Identify **which banner design** achieves the **highest click-through rate (CTR)**.

CTR formula:

$$\text{CTR} = \frac{\textit{Number of clicks}}{\textit{Number of impressions}}$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case study – online advertising

Every time a banner is displayed:

The **advertiser's algorithm** selects which ad to show (via API).

The algorithm **observes the outcome**:

Click → reward = 1

No click → reward = 0

MAB objective:

- Discover the **best-performing ad** early
- **Favor** high-performing ads
- **Eliminate** weak ones quickly

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Case study – online advertising

In this case:

Outcome is **binary** (click / no click)

Can be modeled using a **Bernoulli distribution**

Bernoulli distribution parameter:

$$p = P(click) = CTR\ of\ the\ ad$$

Each ad version has its own (unknown) **click probability** p_i.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Comparison with Previous Example

Setting	Reward Type	Distribution	Example
Slot machines	Continuous	Normal	Random payout amount
Online ads	Binary	Bernoulli	Click (1) or no click (0)

In MAB, *reward modeling changes with the application context*

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

A/B/n testing

- One of the most common exploration strategies is what is called **A/B testing**, which is a method to determine which one of the two alternatives (of online products, pages, ads, and so on) performs better.

- In our case, we have more than two ad versions. So, we will implement what is called **A/B/n testing.**

- We will use A/B/n testing as our baseline strategy for comparison with the more advanced methods that we will introduce afterward. Before going into the implementation, we need to define some notation that we will use throughout this lecture.

Notation

- We will need to keep track of some quantities related to a particular action (the ad chosen for display), a.
- First, we denote the reward (that is, 1 for a click, 0 for no click) received after selecting the action, a, for the ith time by R_i.
- The average reward observed before the nth selection of this same action is defined as follows:

$$Q_n \triangleq \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}$$

  This estimates the expected value of the reward that this action yields, R, after $n-1$ observations.
- This is also called the **action value** of a. Here, this is Q_n estimates of the action value after selecting this action $n-1$ times.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Notation

- Now, we need a bit of simple algebra and we will have a very convenient formula to update the action values

$$Q_{n+1} = \frac{R_1 + R_2 + \cdots + R_n}{n}$$

$$= \frac{R_1 + R_2 + \cdots + R_{n-1}}{n} + \frac{R_n}{n}$$

$$= \frac{n-1}{n-1} \cdot \frac{R_1 + R_2 + \cdots + R_{n-1}}{n} + \frac{R_n}{n}$$

$$= \frac{n-1}{n} \cdot \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1} + \frac{R_n}{n}$$

$$= \frac{n-1}{n} \cdot Q_n + \frac{R_n}{n}$$

$$= Q_n + \frac{1}{n} \cdot (R_n - Q_n)$$

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Notation

- Remember that Q_n is our estimate for the action value of a before we take it for the nth time. When we observe the reward, R_n, it gives us another signal for the action value. We don't want to discard our previous observations, but we also want to update our estimate to reflect the new signal.

  So, we adjust our current estimate, Q_n, in the direction of the **error** that we calculate based on the latest observed reward, $R_n - Q_n$, with a **step size** of $1/n$ and obtain a new estimate, Q_{n+1}. This means, for example, if the latest observed reward is greater than our current estimate, we revise the action value estimate upward.

- For convenience, we define $Q_0 \triangleq 0$.

- Notice that the rate at which we adjust our estimate will get smaller as we make more observations due to the $1/n$ term. So, we put less weight on the most recent observations and our estimate for the action value for a particular action will settle down over time.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Notation

- However, this might be a disadvantage if the environment is not stationary but is changing over time. In those cases, we would want to use a step size that does not diminish over time, such as a fixed step size of $\alpha \in (0,1)$.
- Note that this step size must be smaller than 1 for the estimate to converge (and larger than 0 for a proper update).
- Using a fixed value for α will make the weights of the older observations decrease exponentially as we take action α more and more.

Notation

Let's bring α back to the notation, so we can obtain our formula to update the action values:

$$Q_{n+1}(a) = Q_n(a) + \alpha(R_n(a) - Q_n(a))$$

Here, α is a number between 0 and 1. For stationary problems, we usually set $\alpha = 1/N(a)$, where $N(a)$ is the number of times the action a has been taken up to that point (which was denoted by n initially). In stationary problems, this will help action values converge quicker, due to the diminishing $1/N(a)$ term, rather than chasing after noisy observations.

*Note the difference between **alpha** and **a** symbols!*

Online advertising scenario with A/B/n Test

- Continue to see '**Multi-armed bandits.ipynb**' file in the Teams course page.

1. start with creating the variables to keep track of the rewards in the experiment.
2. run the A/B/n test

   Remember that we randomly select an ad to display during the test and observe whether it gets a click. We update the counter, the action value estimate, and the average reward observed so far.
3. at the end of the test period, choose the winner as the ad that has achieved the highest action value
4. run the best ad identified in the A/B/n test in production.

   At this stage, we don't explore any other actions. So, the incorrect selection of the ad D will have its impact throughout the production period. We continue to record the average reward observed so far to later visualize the ad campaign performance

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Example output

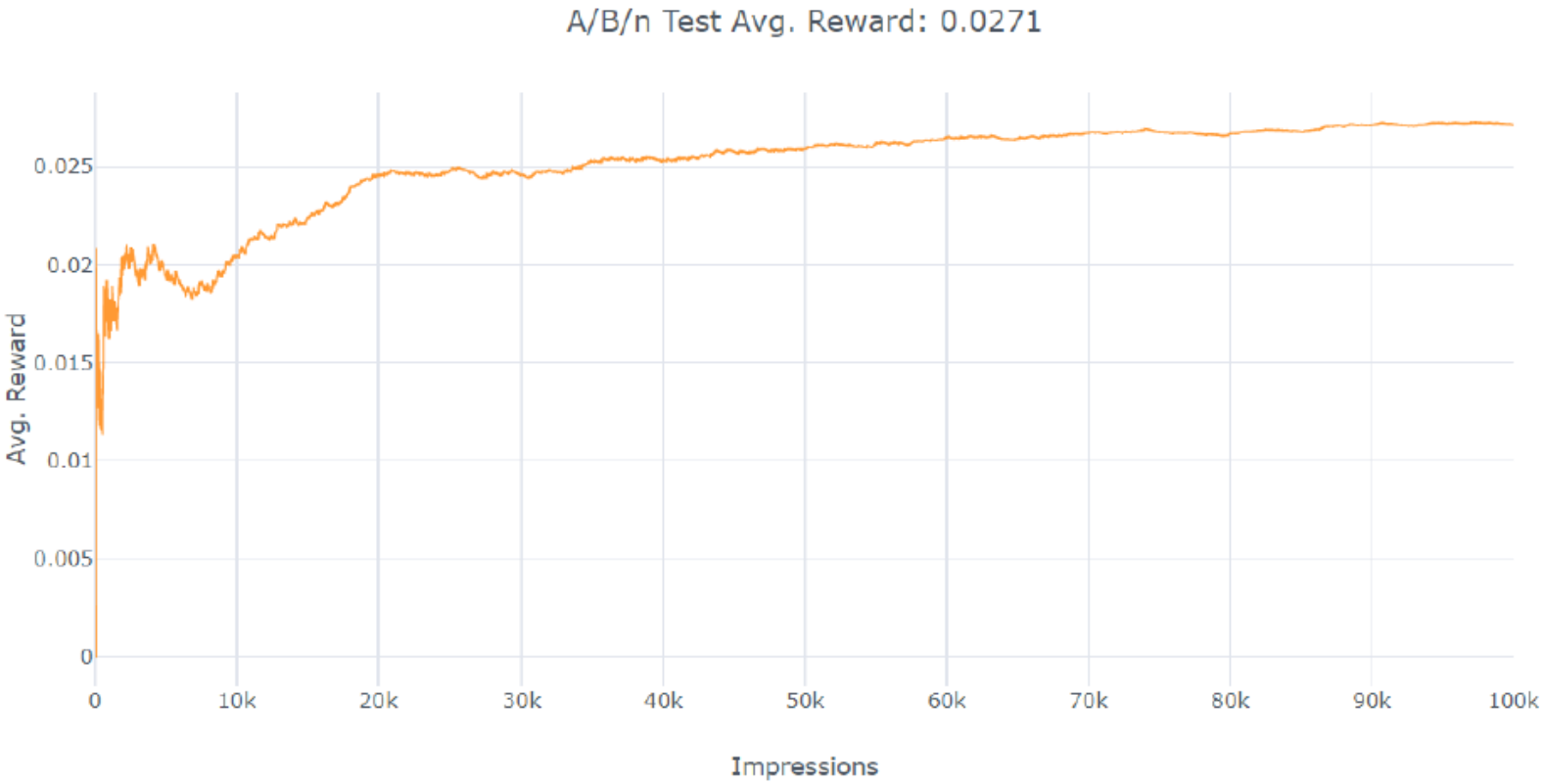


Figure 2.2 – A/B/n test rewards

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

A/B/n Testing: Advantages and Disadvantages

Limitation 1: Inefficient and Static

Does **not learn dynamically** from new observations.

Keeps **pre-determined probabilities** throughout the test.

Fails to promote or eliminate options early — even when results are obvious.

Misses opportunity to improve performance during the test.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

A/B/n Testing: Advantages and Disadvantages

Limitation 2: Irreversible Decision

Once the test ends, the **selected alternative stays fixed**.

If chosen incorrectly (due to short test duration or noise):
→ The **wrong choice persists** in production.

No mechanism to correct decisions after deployment.

A/B/n Testing: Advantages and Disadvantages

Limitation 3: No Adaptation to Change

Cannot handle **non-stationary environments**.

If user behavior or reward probabilities **change over time**,
→ the method **fails to detect or adapt**.

Static once deployed = poor long-term performance.

A/B/n Testing: Advantages and Disadvantages

Limitation 4: Test Duration Trade-Off

The **length of the test** (hyperparameter) is critical.

Too **short** → wrong option may seem best (due to noise).

Too **long** → wasted resources on inferior options.

Balancing duration and accuracy is **non-trivial**.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

A/B/n Testing: Advantages and Disadvantages

Advantage: Simplicity

Despite drawbacks, A/B/n testing remains:

- **Simple and intuitive**
- **Easy to implement**
- **Widely used in industry** for quick comparisons

However:
It's a **naive** exploration method for MAB problems — not adaptive or optimal.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

ε-greedy actions

Key Idea:
A simple, effective, and widely used method to address the **exploration-exploitation trade-off**.

How it works:

With probability **1 − ε** → **exploit**: choose the action with the **highest observed reward so far**

With probability **ε** → **explore**: choose a **random action** regardless of past rewards

Notes:

ε is between 0 and 1, usually **small** (e.g., 0.1)

Ensures **continuous exploration** throughout the experiment

Balances learning new information with leveraging known good options

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Online advertising scenario with ε-greedy

- Continue to see ‘**Multi-armed bandits.ipynb**’ file in the Teams course page.

Advantages and disadvantages of ε-greedy actions

Similarities with A/B/n testing:

Both are somewhat **inefficient and static** in allocating exploration.

Clearly bad actions are **not written off early**.

Exploration budget is **fixed**, not adjusted if an action is under- or over-explored.

Advantages and disadvantages of ε-greedy actions

Advantages Over A/B/n Testing:

Continuous exploration: Unlike A/B/n testing, ε-greedy keeps exploring throughout.

Can **adapt to non-stationary environments** by picking up changes in action performance.

In stationary environments, performance is **similar to A/B/n**, except for ongoing exploration.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Advantages and disadvantages of ε-greedy actions

Potential Improvements:

Dynamic ε (epsilon) scheduling:

Start with a **high ε** → explore more initially.

Gradually **decrease ε** → exploit more later.

Weighted or recent-focused rewards:

Instead of simple averages for Q-values, **emphasize recent observations**.

Helps adapt faster to changing environments.

Contains materials from "Mastering Reinforcement Learning with Python, Enes Bilgin, Packt Publishing, 2020."

Advantages and disadvantages of ε-greedy actions

Warning:

Introducing dynamic ε or weighted Q-values adds **hyperparameters**.

Tuning these parameters is critical; wrong values may **worsen performance**.

Trade-off between **flexibility/adaptivity** and **simplicity**.